2024 13th Mediterranean Conference on Embedded Computing (MECO) | 979-8-3503-8756-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/MECO62516.2024.10577852

# Smart Contract Vulnerability Detection Using Deep Learning Algorithms on EVM bytecode

Lejdi Prifti
Department of Computer Engineering, Epoka University
Tirana, Albania
lprifti21@epoka.edu.al

Betim Cico
Department of Computer Engineering, Epoka University
Tirana, Albania
bcico@epoka.edu.al

Dimitrios Karras
Department of Computer Engineering, Epoka University
Tirana, Albania
dkarras@epoka.edu.al

Abstract—**In the quickly changing world of blockchain technology, it is critical to guarantee the security of the self-executing contracts, written in programming languages like Solidity called smart contracts. Not all security vulnerabilities in smart contracts will be found by human code reviews and security audits using traditional methods. Deep learning networks have become a promising answer to this problem. In this paper, we present the architecture of two models—using convolutional and recurrent neural networks—that are intended to effectively discover five vulnerabilities in smart contracts. To train and validate the models, we used a dataset that includes 106474 audited smart contracts taken from the public Ethereum blockchain. Instead of the source code that is typically used by most deep learning-based solutions, the models receive input in the form of Ethereum Virtual Machine (EVM) bytecode. Across all five vulnerabilities, the Recurrent Neural Network model has an average micro F1-score of 0.93, whereas the Convolutional Neural Network achieves an average micro F1-score of 0.89. Through comparative research with various deep learning systems and static analysis tools, we have determined that EVM bytecode may be leveraged as a feature to detect vulnerabilities in smart contracts.**

***Keywords—Deep Learning, Smart Contracts, Security***

## I. Introduction

Deep learning, a branch of machine learning, has achieved good results in several fields, including speech recognition, computer vision, and natural language processing.

In this paper, we present the architecture of two deep learning neural networks that are fed the EVM bytecode of the smart contracts, whose characters are split into pairs. Although our models have a relatively small number of parameters, they achieve good performance in detecting vulnerabilities into Ethereum smart contracts.

Through the literature review we have provided a comprehensive overview of the methods currently used to find smart contract vulnerabilities and we have classified them into two groups: deep learning methods and static analysis tools. The importance of several well-known tools in enhancing the security of Ethereum smart contracts is highlighted, including Oyente, Mythril, Maian, Zeus, EtherFuzz, and Securify. These technologies do, however, have several drawbacks, such as restricted vulnerability coverage, false positives and false negatives.

Several deep learning models and methods, including Convolutional Neural Networks (CNNs), Long-Short Term Memory (LSTM) networks, and neuro-symbolic frameworks have been proven to be efficient.

However, most of these models expect to input the source code of smart contracts. Instead of the source code, we use the EVM bytecode. From the results we received, we have concluded that EVM bytecode split into pairs is a better feature to detect vulnerabilities in smart contracts. We have compared the results of our model with those achieved by Rossini et al. in the same dataset. For future work, we suggest expanding data collection efforts by increasing the number of smart contract vulnerabilities. Furthermore, we recommend increasing convolutional block complexity, as well as creating a single model that expects two forms of inputs, the source code, and the bytecode of the smart contract.

## II. Background

Smart contracts are programs that run on the Ethereum blockchain. They consist of data and code that is kept on the Ethereum blockchain at a particular address. Nonetheless, smart contracts remain a specific type of Ethereum account known also as contract accounts. These types of accounts have a balance and can be the target of transactions. They are deployed to the network and run as programmed. Submitting transactions that carry out a function specified on the smart contract is how user accounts can then communicate with the contract. Similar to a traditional contract, a smart contract can specify rules and have the code automatically enforce them. Smart contract interactions are irreversible and cannot be deleted by default. The four stages of a smart contract, displayed in Figure 1, are creation, deployment, execution, and completion [47].

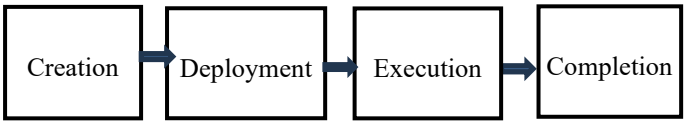


Figure 1. The lifecycle of Ethereum smart contracts.

### A. Creation Phase

During the creation phase, the smart contract is written in a high-level programming language, such as Solidity. The developer specifies the rules that will guide the operations.

979-8-3503-8756-8/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:39:00 UTC from IEEE Xplore. Restrictions apply.

### B. Deployment Phase

The primary objective of the deployment phase is to transfer the rules defined in the smart contract during the creation phase to the EVM. However, the Ethereum Virtual Machine (EVM) understands a low-level, stack-based bytecode language as shown in Figure 2.

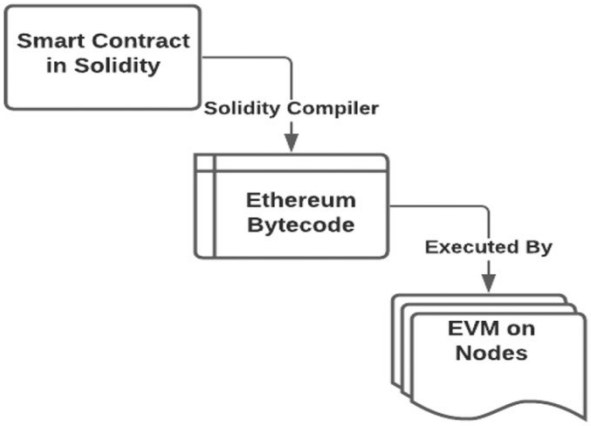


Figure 2. The workflow of Ethereum smart contract source. [46]

Consequently, the smart contract must be compiled into opcodes, which are the EVM's low-level instructions. The EVM bytecode is a set of instructions that the Ethereum nodes execute. It is a low-level representation of smart contract logic. Each bytecode instruction corresponds to a specific operation, such as arithmetic, storage manipulation, conditional branching, etc. The following is an example bytecode of a simple counter smart contract.

*0x6060604081815260018152602060048201526 01f6024820 1527f48656c6c6f2c20576f726c642100000….*

The developer starts a transaction that includes the bytecodes kept in the transaction structure's "init" field during the Deployment phase.

### C. Execution Phase

When a smart contract is deployed to the Ethereum blockchain, the EVM ensures the execution of the contract's bytecode in a decentralized and deterministic manner across all nodes in the network. Until the task is completed or the gas limit is reached, the EVM carries out each instruction one at a time. The moment a new block is mined, this process takes place.

### D. Completion Phase

In the completion phase, after the transaction has been concluded, states are modified and documented in blockchains. The new block is mined and distributed to all the nodes in the network. Smart contract interactions are irreversible and cannot be deleted from the ledger.

This section provides a brief explanation of the vulnerabilities present in Ethereum smart contracts. These types of vulnerabilities were trained into our models for detection and classification.

### A. Reentrancy

Reentrancy refers to a scenario in which contract A calls contract B, but because of Solidity's fallback mechanism, contract B may call A back and execute A's call once more. In the event that calls from other contracts are unable to locate a matching function, the fallback function will be called. When the caller uses the call function without passing in a function signature, the callee's fallback function will be triggered. This function may therefore call the caller's function to re-enter the caller. If not properly managed, reentrant calls can allow malicious contracts to manipulate the state of the calling contract in unintended ways or siphon off funds.

### B. Arithmetic Issues

When performing addition, subtraction, or storing user input with integer variables that contain value limitations, overflow/underflow can occur. If the value of variables goes beyond the upper or lower bound, the value will wrap to the other side of the bound.

### C. Unchecked external calls

Unchecked external calls in Solidity are instances in which calls to external functions are made without properly implementing error handling methods or verifying the return values. In Solidity, external calls are interactions with external addresses or other contracts. If not used appropriately, they can result in several problems.

### D. Access control

“tx.origin” is always an external account that keeps track of the originating caller's address for a transaction. Bob is “tx.origin” if contract A is being called by contract B, and contract A is being called by Bob. But contract A's msg.sender is contract B's. The person who called right away is identified as “msg.sender”. It is advised never to utilize “tx.origin” for authentication or identity verification.

### E. Other issues (i.e. freezing ether, uninitialized local, etc.)

A smart contract's function is to send and receive ether. A person is considered to be in a freezing-ether state if they can only accept ether and are unable to send ether out. The contract might be avaricious and freeze the Ether transferred to its address if it does not mention any withdrawal functions. Additionally, an uninitialized local refers to a variable that has been declared but not initialized with a value. Other types of vulnerabilities, such as mapping deletion, uninitialized state, uninitialized storage, etc. are grouped into the class named “Other”.

## III. Literature Review

In Table I, we give an overview of the current approaches to smart contract vulnerability detection. We will focus more on some of these methods.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:39:00 UTC from IEEE Xplore. Restrictions apply.

There are several well-known solutions based on symbolic execution that are widely used in the field of automated security tools for smart contracts.

Oyente is a security analysis tool designed to identify potential flaws and vulnerabilities in Ethereum smart contracts. It uses symbolic execution techniques to simulate the behavior of the contract without executing it, allowing it to identify intricate flaws that could be difficult to find through manual analysis.

Mythril is another popular open-source tool for identifying security flaws in Ethereum smart contracts code. However, it has drawbacks such as potential false positives or negatives and the complexity of the contract code.

Maian, introduced by Ivica Nikolic et al., identifies potential greedy, prodigal, and suicidal smart contracts. After analyzing 970,898 contracts, it was found that 97% of prodigal, 97% suicidal, and 69% greedy contracts were true positives. [1, 2, 3]

Securify was introduced by Peter et al [8]. It examines every contract action to prevent unfavorable false negatives.

TABLE I. SUMMARY OF REFERENCES BASED ON THE METHODOLOGY USED

| Method | References |
|---|---|
| Symbolic Execution | [1, 2] |
| Game Theory & Probabilistic Model Testing | [4] |
| Concrete Validation and Analysis | [3, 8, 9, 12] |
| LLVM Framework and Type Systems | [5, 7] |
| Dynamic Linearizability Checker | [6] |
| Deep Learning | [10, 11, 13, 15, 16, 23] |
| Ensemble Learning | [21, 26] |
| Multi-Modal Decision Fusion | [19, 20] |
| Sequence Learning | [14] |
| Transformer Model | [33, 34] |
| Graph Neural Networks (GNNs) | [30, 32, 35, 36, 39, 40, 42] |
| AST-based Approaches | [24, 25, 26, 42, 43] |
| Other Approaches | [27, 28, 31, 37, 38, 41, 44, 45, 46] |

Deep learning has led to the development of neural networks to assist in detecting vulnerabilities in smart contracts.

By combining metric learning triplet loss with a bidirectional long short-term memory (BiLSTM) network model, Wang Meiying et al. proposed a unique vulnerability detection method for Ethereum smart contracts [11].

In their study, Oliver Lutz et al. proposed ESCORT with the goal of overcoming the scalability and generalization restrictions of the earlier published research. ESCORT is a multi-output neural network that has been trained to recognize each specific vulnerability class in addition to learning the bytecode properties [13]. ESCORT makes use of a feature extractor to accomplish these two objectives since it can read contract bytecode regardless of its flaws and extract its semantic and syntactic information.

The study by Wesley Joon-Wie Tann et al. suggests using a sequence learning approach [14]. The authors expressed the contract's opcode specifically using the one-hot encoding and embedding matrix. An LSTM model is trained to predict if a given smart contract is safe or vulnerable using the obtained code vectors as input (binary classification). The LSTM-based technique cannot distinguish between the different vulnerability categories.

Zhuang et al. proposed a GNN-based technique for contract graphs, normalizing nodes and edges for crucial function calls and variables. The technique is limited in applicability and efficacy, with an F1-score of 77% for all three vulnerabilities.

Merity et al. adopts the "Average Stochastic Gradient Descent Weighted Dropped LSTM" for regularizing the LSTM models.

Weichu Deng et al. developed a multimodal decision fusion method for smart contract vulnerability detection, considering control structure and code semantics. [15, 16, 19]

Zhang et al. [21] proposed leveraging ensemble learning (EL) techniques to smart contract vulnerabilities prediction. Multiple NNs were included in the proposed approach. These models were then used to create an ensemble framework known as the Information Graph and Ensemble Learning-based Smart Contract Vulnerability Detection technique (SCVDIE).

Huang et al. [22] developed a smart contract vulnerability model using multi-task learning. The model's detection capabilities were enhanced to enable the identification and detection of vulnerabilities by assigning supplementary tasks to learn more directional vulnerability features.

Two novel model families have been developed by Hellendoorn et al. [28] that efficiently combine the longer distance information that the sequence model can convey with the semantic structure information that the GGNN can access.

Graph Relational Embedding Attention Transformer (GREAT) is another family of families that conveys structural relations instead by generalizing the relative position embeddings in Transformers by Shaw et al. [29].

## IV. RESEARCH GAP

Based on the literature review and previous works, we have identified that there has been relatively little research on multi-class categorization about Ethereum smart contract

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:39:00 UTC from IEEE Xplore. Restrictions apply.

vulnerabilities based on Ethereum Virtual Machine (EVM) bytecode.

The question of whether EVM bytecode could be used as a feature for uncovering smart contract vulnerabilities has been answered in the work of Rossini et al.

In our work, we aim at developing two models specifically using RNNs and CNNs and train them using the EVM bytecode as input. Finally, we are going to compare the performance of each compared to the previous approaches developed by Rossini et al. The performance of each model and tool is evaluated based on the weighted and micro F1-score. Our goal is to point out how well these models trained on EVM bytecode are able to capture patterns in detecting and classifying five different types of vulnerabilities.

## IV. Dataset

The dataset used to train and validate the models was created by Rossini et al. [17, 48] and is publicly accessible from the HuggingFace hub. The dataset consists of verified Ethereum smart contracts provided by SmartBugs, ScrawlID and Smart Contract Sanctuary, which is a repository in Github that collects smart contracts in different blockchain environments. The authors of the dataset then used the Slither contract flattener to download the source code from the repository or via Etherscan [48]. Using INFURA as their endpoint, they used the web3.eth.getCode() function of the Web3.py module to extract the bytecode. Finally, the Slither static analysis tool was used to review every smart contract. After identifying 38 distinct vulnerability types in the gathered contracts, they were categorized into five groups. These groups represent five different sorts of vulnerabilities: reentrancy, arithmetic issues, unchecked calls, access control, and others. Others is a class that groups all other detectors [48].

The dataset contains 106474 audited smart contracts split into three categories: train, validation, and test. The train split contains 79641 entries, the test split 15972 entries and the validation split 10861 entries. Every data object has the address, source code, bytecode, and slither audited output. The address is a representation of the Ethereum blockchain platform's smart contract address. The source code is its plain-text code in Solidity. The low-level, machine-readable hexadecimal code known as bytecode on Ethereum, explains how a smart contract functions and behaves. Slither consists of a list of vulnerabilities detected by the Slither static analysis tool.

## V. Architectural Models Of Our Solution

Both models receive the input EVM bytecode, which is divided into pairs of characters. This division by pairs allows for a more accurate representation of the Ethereum opcodes. Contrary to other models that are fed with source code, we feed these sequences of pairs from the EVM bytecode into our models. Both models are trained and validated on a GPU T4 with 50 GB of System RAM and 15 GB of GPU RAM.

### *A. Convolutional Neural Networks*

The model is a multi-output convolutional neural network (CNN) as shown in Figure 3. The text vectorizer layer uses 256 tokens as the maximum number of tokens, an output sequence length of 21041 and splits by whitespace. The embedding layer uses an input dimension of 256, an input length of 21041 and an output dimension of 128. The model has three sets of convolutional blocks, each consisting of two 1D convolutional layers followed by a max-pooling layer. These convolutional layers are designed to capture different levels of abstraction in the input data.

- First Block: 2 convolutional layers with 4 filters each, kernel size of 3, stride of 1, followed by max pooling.
- Second Block: 2 convolutional layers with 8 filters each, kernel size of 3, stride of 1, followed by max pooling.
- Third Block: 2 convolutional layers with 16 filters each, kernel size of 3, stride of 1, followed by max pooling.

After each convolutional block, the number of features is doubled to give the model the chance to forget any of the patterns it may have previously learnt, as suggested also in the WaveNet [18]. At the end, there is a global max-pooling layer to capture the most important features from each feature map.

A dense (fully connected) layer follows the global max-pooling layer. It has 32 units and uses the ReLU activation function. There is a distinct dense layer with a single unit and sigmoid activation for each class (indexed by an index). The loss function used is binary cross entropy, the learning rate is 0.001 and the optimization function is Adam.

The model is fitted on the training dataset and trained for 35 epochs. It has in total 36557 trainable parameters and each epoch takes about 165 seconds to run.

### *B. Recurrent Neural Networks*

The model architecture with RNNs is composed of various layers that sequentially process input data and produce a multi-output structure as shown in Figure 4. The text vectorizer layer splits by whitespace, uses an output sequence length of 21041 characters, and allows a maximum of 256 tokens. The embedding layer uses a 256-dimensional input and output, with an input length of 21041. Two Gated Recurrent Unit (GRU) layers are employed for sequential modeling of the embedded text data.

The first GRU layer has 64 units and is configured to return sequences, while the second GRU layer has 32 units. The activation function employed is the hyperbolic tangent. A dropout layer with a dropout rate of 0.2 is introduced to prevent overfitting, followed by a fully connected dense layer with 16 units and a hyperbolic tangent activation function. The final output layer is made up of several sub-layers, each of which uses sigmoid activation to produce a binary classification result.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:39:00 UTC from IEEE Xplore. Restrictions apply.

The output sub-layer count is equal to the five labels that are supplied. The loss function used is binary cross entropy, the learning rate is 0.001 and the optimization function is Adam.

The model is fitted on the training dataset and trained for 20 epochs. It has 80037 trainable parameters, and each epoch takes about 2216 seconds.

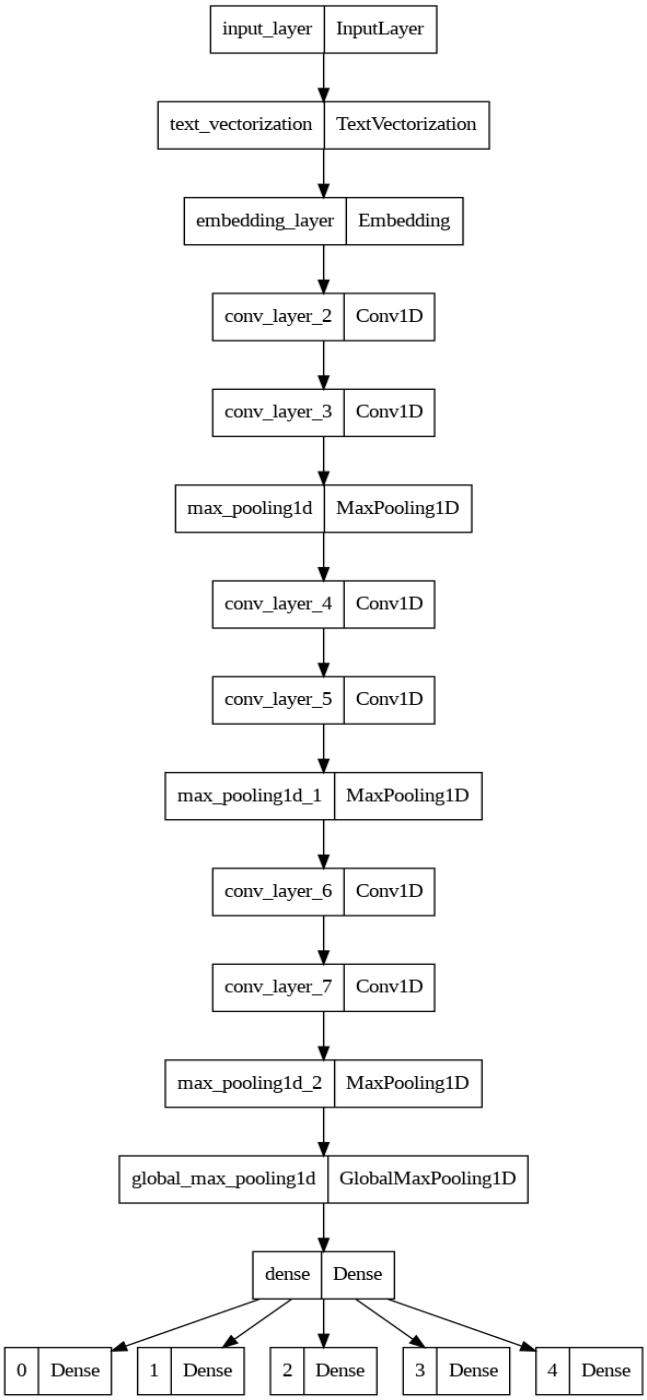


Figure 3. Model architecture with CNNs.

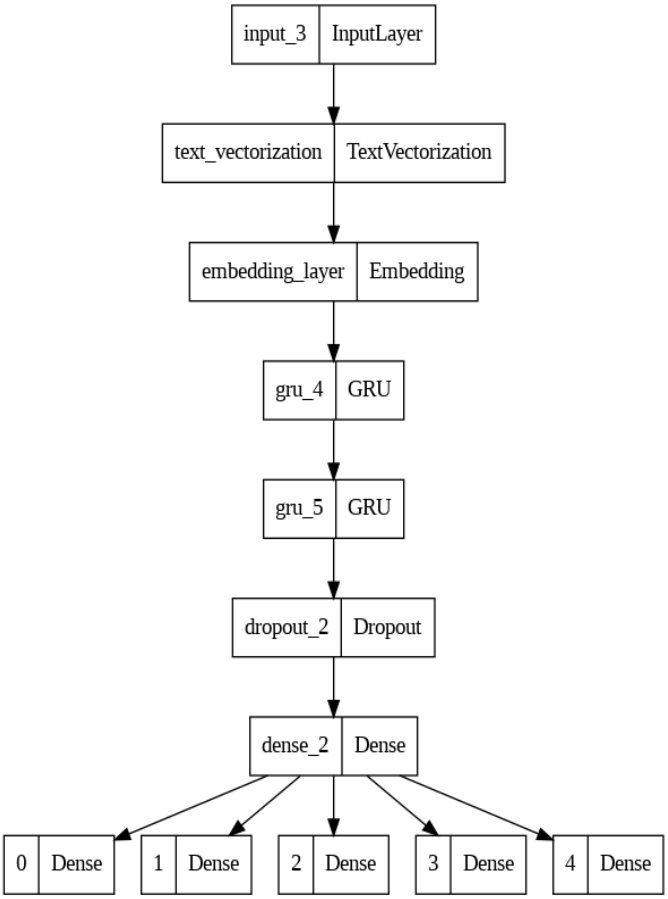


Figure 4. Model architecture with RNNs.

## VI. RESULTS

This section summarizes the results we got during the testing phase. Table II shows the results received on the test split of the dataset.

The micro-F1 score for both our models is an average micro-F1 score calculated across all five vulnerability classes on the validation split of the dataset. Table 3, as well as Figure 5, displays the micro F1-score that the different models trained by Rossini et al. [17] achieved on the validation split of the dataset.

By looking at the table, we can see that our models perform better. This could be due to the following reasons. We use a longer sequence of input, as well as feeding the input split into pairs. Furthermore, we would argue that the architecture of the model plays an important role.

## VII. CONCLUSION

Based on the results achieved during testing, we conclude that EVM bytecode is effective in showing smart contract vulnerabilities. We base our conclusion on the fact that we managed to achieve an average micro F1-score of 0.93 and 0.89 across all five vulnerabilities for our RNN model and CNN model, respectively. Additionally, we have observed that the RNN model performs better than the CNN model.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:39:00 UTC from IEEE Xplore. Restrictions apply.

For future initiatives, we recommend stepping up data gathering efforts and convolutional block complexity in the model architecture. The quantity and diversity of the dataset can be increased to facilitate the model's generalization and provide more detailed insights and applications. The auditing of smart contracts could be performed by experts in the field to ensure the correct labeling of the smart contracts. Finally, we encourage developing a concatenation between two different types of inputs in a single model. One input layer would expect the bytecode of the smart contract and the other input layer would expect the source code of the same smart contract.

TABLE II. METRICS ON THE TEST DATASET

| **Vulnerability** | | **Access control** | | **Arithmetic issues** | |
|---|---|---|---|---|---|
| **Archit. model** | | CNN | RNN | CNN | RNN |
| **Accuracy (%)** | | 90.515 | 93.185 | 89.510 | 92.337 |
| **Precision** | | 0.899 | 0.928 | 0.891 | 0.920 |
| **Recall** | | 0.905 | 0.931 | 0.895 | 0.923 |
| **Weighted F1-score** | | 0.901 | 0.928 | 0.881 | 0.921 |
| **Other issues** | | **Reentrancy** | | **Unchecked external calls** | |
| CNN | RNN | CNN | RNN | CNN | RNN |
| 84.730 | 88.945 | 88.267 | 91.369 | 90.157 | 96.815 |
| 0.842 | 0.887 | 0.881 | 0.913 | 0.901 | 0.968 |
| 0.847 | 0.889 | 0.8 82 | 0.913 | 0.901 | 0.968 |
| 0.837 | 0.884 | 0.879 | 0.912 | 0.901 | 0.968 |

TABLE III. COMPARISON OF F1-SCORE BETWEEN DIFFERENT MODELS

| **Model / Tool** | **Micro F1-score** |
|---|---|
| ResNet1D | 0.8381 |
| ResNet | 0.7928 |
| Inception | 0.8015 |
| LSTM Baseline | 0.7953 |
| Our RNN model | 0.93 |
| Our CNN model | 0.89 |

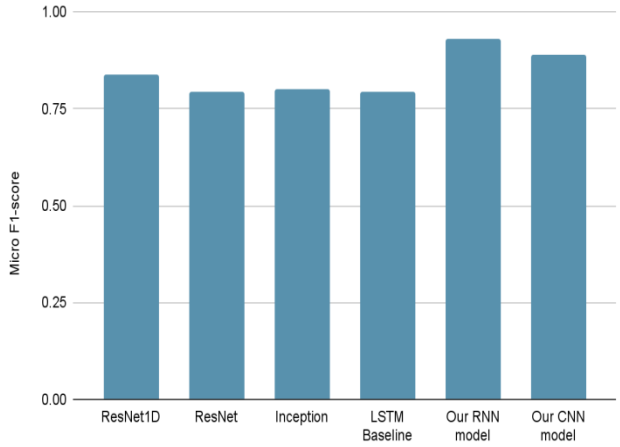


Figure 5. Comparison of models

## REFERENCES

[1] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making Smart Contracts Smarter. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). 254–269.

[2] Mythril. (201.8), from: https://github.com/ConsenSys/mythril.

[3] Ivica Nikolic, Aashish Kolluri, Ilya Sergey, Prateek Saxena, and Aquinas Hobor. 2018. Finding the Greedy, Prodigal, and Suicidal Contracts at Scale.

[4] Giancarlo Bigi, Andrea Bracciali, Giovanni Meacci, and Emilio Tuosto. 2015. Validation of Decentralised Smart Contracts Through Game Theory and Formal Methods. In Programming Languages with Applications to Biology and Security.

[5] Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Anitha Gollamudi, Georges Gonthier, Nadim Kobeissi, Natalia Kulatova, Aseem Rastogi, Thomas Sibut-Pinote, Nikhil Swamy, and Santiago Zanella-Béguelin. 2016. Formal Verification of Smart Contracts: Short Paper. In Proceedings of the 2016 ACM Workshop on Programming Languages and Analysis for Security (PLAS). 91–96.

[6] Shelly Grossman, Ittai Abraham, Guy Golan-Gueta, Yan Michalevsky, Noam Rinetzky, Mooly Sagiv, and Yoni Zohar. 2018. Online detection of effectively callback free objects with applications to smart contracts. PACMPL 2, POPL (2018), 48:1–48:28.

[7] Sukrit Kalra, Seep Goel, Mohan Dhawan, and Subodh Sharma. 2018. ZEUS: Analyzing Safety of Smart Contracts.

[8] Petar Tsankov, Andrei Dan, Dana Drachsler-Cohen, Arthur Gervais, Florian Bünzli, Martin Veche. 2018. Securify: Practical Security Analysis of Smart Contracts.

[9] Lexi Brent, Neville Grech, Sifis Lagouvardos, Bernhard Scholz, and Yannis Smaragdakis. 2020. Ethainter: A Smart Contract Security Analyzer for Composite Vulnerabilities.

[10] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph neural network," ACM Trans. Softw. Eng. Methodol., vol. 30, Apr. 2021.

[11] Wang Meiying, Zheyu Xie, Xuefan Wen, Jianmin Li, and Kuanjiu Zhou. 2023. Ethereum Smart Contract Vulnerability Detection Model Based on Triplet Loss and BiLSTM.

[12] Xiaoyin Wang, Jiaze Sun, Chunyang Hu, Panpan Yu, Bin Zhang, Donghai Hou. 2022. EtherFuzz: Mutation Fuzzing Smart Contracts for TOD Vulnerability Detection.

[13] Oliver Lutz, Huili Chen, Hossein Fereidooni, Christoph Sendner, Alexandra Dmitrienko, Farinaz Koushanfar, Ahmad Reza Sadeghi. 2023. ESCORT: Ethereum Smart COntRacTs Vulnerability Detection using Deep Neural Network and Transfer Learning.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:39:00 UTC from IEEE Xplore. Restrictions apply.

[14] Wesley Joon-Wie Tann, Xing Jie Han, Sourav Sen Gupta, and Yew-Soon Ong. 2018. Towards safer smart contracts: A sequence learning approach to detecting security threats.

[15] Yuan Zhuang, Zhenguang Liu, Peng Qian, Qi Liu, Xiang Wang, and Qinming He. 2020. Smart Contract Vulnerability Detection Using Graph Neural Networks.

[16] Stephen Merity, Nitish Shirish Keskar, and Richard Socher. 2017. Regularizing and optimizing LSTM language models.

[17] Martina Rossini, Mirko Zichichi, Stefano Ferret, Smart Contracts Vulnerability Classification Through Deep Learning, 2022.

[18] WaveNet: A Generative Model for Raw Audio, Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, 2016

[19] Deng, W.; Wei, H.; Huang, T.; Cao, C.; Peng, Y.; Hu, X. Smart Contract Vulnerability Detection Based on Deep Learning and Multimodal Decision Fusion. Sensors 2023, 23, 7246. https://doi.org/ 10.3390/s23167246

[20] Gao, Z.; Jiang, L.; Xia, X.; Lo, D.; Grundy, J. Checking Smart Contracts With Structural Code Embedding. IEEE Trans. Softw. Eng. 2021, 47, 2874–2891.

[21] Zhang, L.; Wang, J.; Wang, W.; Jin, Z.; Zhao, C.; Cai, Z.; Chen, H. A Novel Smart Contract Vulnerability Detection Method Based on Information Graph and Ensemble Learning. Sensors 2022, 22, 3581. https://doi.org/10.3390/s22093581

[22] Huang, J.; Zhou, K.; Xiong, A.; Li, D. Smart Contract Vulnerability Detection Model Based on Multi-Task Learning. Sensors 2022, 22, 1829. https://doi.org/10.3390/ s22051829

[23] Fan, Y.; Shang, S.; Ding, X. Smart Contract Vulnerability Detection Based on Dual Attention Graph Convolutional Network. In Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Proceedings of the International Conference on Collaborative Computing: Networking, Applications and Worksharing, Suzhou, China, 16–17 October 2021; Springer: Berlin/Heidelberg, Germany, 2022; pp. 335–351.

[24] Hao Peng, Lili Mou, Ge Li, Yuxuan Liu, Lu Zhang, and Zhi Jin. Building program vector representations for deep learning. InSongmao Zhang, Martin Wirsing, and Zili Zhang, editors, International Conference on Knowledge Science, Engineering and Management (KSEM), pages 547–553, Cham, 2015. Springer International Publishing.

[25] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," 2020.

[26] Ding, Y.; Suneja, S.; Zheng, Y.; Laredo, J.; Morari, A.; Kaiser, G.; Ray, B. VELVET: A novel ensemble learning approach to automatically locate VulnErable sTatements. arXiv 2021, arXiv:2112.10893.

[27] Xuan Xie, Kristian Kersting, Daniel Neider. 2022. Neuro-Symbolic Verification of Deep Neural Networks.

[28] V. J. Hellendoorn, C. Sutton, R. Singh, P. Maniatis, and D. Bieber, "Global relational models of source code," in International Conference on Learning Representations, 2020.

[29] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. arXiv preprint arXiv:1803.02155, 2018.

[30] Y. Wang, K. Wang, F. Gao, and L. Wang, "Learning semantic program embeddings with graph interval neural network," Proc. ACM Program. Lang., vol. 4, Nov. 2020.

[31] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine grained interpretations," in Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2021, (New York, NY, USA), p. 292–303, Association for Computing Machinery, 2021.

[32] D. Tarlow, S. Moitra, A. Rice, Z. Chen, P.-A. Manzagol, C. Sutton, and E. Aftandilian, "Learning to fix build errors with graph2diff neural networks," 2019.

[33] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "A transformer-based approach for source code summarization," in ACL, pp. 4998–5007, 2020.

[34] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in Findings of the Association for Computational Linguistics: EMNLP 2020, (Online), pp. 1536–1547, Association for Computational Linguistics, Nov. 2020.

[35] Mi, F.; Wang, Z.; Zhao, C.; Guo, J.; Ahmed, F.; Khan, L. VSCL: Automating Vulnerability Detection in Smart Contracts with Deep Learning. In Proceedings of the 2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), Virtual Conference, 3–6 May 2021; pp. 1–9, ISBN 166543578X

[36] Wu, H.; Zhang, Z.; Wang, S.; Lei, Y.; Lin, B.; Qin, Y.; Zhang, H.; Mao, X. Peculiar: Smart Contract Vulnerability Detection Based on Crucial Data Flow Graph and Pre-training Techniques. In Proceedings of the 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE), Wuhan, China, 25–28 October 2021; pp. 378–389

[37] Eshghie, M.; Artho, C.; Gurov, D. Dynamic Vulnerability Detection on Smart Contracts Using Machine Learning. In Proceedings of the Evaluation and Assessment in Software Engineering (EASE 2021), Trondheim, Norway, 21–23 June 2021; pp. 305–312.

[38] Elizabeth Dinella, Hanjun Dai, Ziyang Li, Mayur Naik, Le Song, and Ke Wang. 2020. Hoppity: Learning Graph Transformations to Detect and Fix Bugs in Programs. International Conference on Learning Representations (2020).

[39] S. Suneja, Y. Zheng, Y. Zhuang, J. Laredo, and A. Morari, "Learning to map source code to software vulnerability using code-as-a-graph," 2020.

[40] Petr Vytovtov and Kirill Chuvilin. Unsupervised classifying of software source code using graph neural networks. In 2019 24th Conference of Open Innovations Association (FRUCT), pages 518–524. IEEE, 2019.

[41] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. Proceedings of the ACM on Programming Languages, 3(POPL):40, 2019.

[42] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, and Dinh Phung. 2022. ReGVD: Revisiting Graph Neural Networks

[43] David Hin, Andrey Kan, Huaming Chen, and M. Ali Babar. 2022. LineVD: Statement-level Vulnerability Detection using Graph Neural Networks. In MSR '22: Proceedings of the 19th International Conference on Mining Software Repositories, May 23-24, 2022, Pittsburgh, PA, USA. ACM, New York, NY, USA, 12 pages.

[44] Sefa Eren Şahin, Ecem Mine Özyedierler, and Ayse Tosun. 2022. Predicting vulnerability inducing function versions using node embeddings and graph neural networks. Information and Software Technology 145, (May 2022), 106822.

[45] Evangelos, D., Katsadouros and Charalampos, Z., Patrikakis. 2022. A Survey on Vulnerability Prediction using GNNs. 26th Pan-Hellenic Conference on Informatics (PCI 2022), November 25-27, 2022, Athens, Greece. ACM, New York, NY, USA, 9 Pages.

[46] Toward Blockchain-Enabled Supply Chain Anti-Counterfeiting and Traceability – Scientific

[47] V. Buterin. 2013. Ethereum white paper: a next generation smart contract & decentralized application platform.

[48] M. Rossini, Slither Audited Smart Contracts Dataset, 2022. Available at: [https://huggingface.co/datasets/mwritescode/slither-audited-smart-contracts]


Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:39:00 UTC from IEEE Xplore. Restrictions apply.